

rand(), rand_r() e falso compartilhamento na estimativa de π

Matheus Marinho

9 de setembro de 2026

1. Objetivo e método

O objetivo foi estimar π por Monte Carlo e comparar quatro implementações OpenMP, combinando dois geradores pseudoaleatórios e dois métodos de acumulação. Cada amostra é um ponto uniforme (x, y) no quadrado $[0, 1] \times [0, 1]$. O ponto é um acerto quando $x^2 + y^2 \leq 1$; logo:

$$\hat{\pi} = 4 \cdot \text{acertos} / \text{número total de pontos}$$

As versões foram executadas com a mesma quantidade de pontos, número de threads e semente. O tempo de parede foi medido com `omp_get_wtime()`. A soma serial final do vetor faz parte do intervalo cronometrado; somente a inicialização e a impressão ficaram de fora. A soma possui apenas uma parcela por thread e custo desprezível diante dos dez milhões de amostras.

2. As quatro versões

2.1 rand() com contador privado e critical

Cada thread conta os acertos em uma variável local. No fim, entra uma única vez na região crítica para acrescentar seu subtotal à variável global:

```
#pragma omp parallel default(none) shared(pontos, total_global)
{
    long long acertos_locais = 0;

    #pragma omp for schedule(static)
    for (long long i = 0; i < pontos; i++) {
        double x = coordenada(rand());
        double y = coordenada(rand());
        acertos_locais += (x * x + y * y <= 1.0);
    }

    #pragma omp critical
    total_global += acertos_locais;
}
```

O contador local normalmente permanece em registrador ou cache privado. Como há apenas uma entrada no `critical` por thread, o custo dessa acumulação é $O(T)$, onde T é o número de threads, e não $O(N)$ para N pontos.

2.2 rand() com vetor compartilhado

Cada thread incrementa `acertos_por_thread[id]`. Não existe condição de corrida, pois cada posição tem um único escritor. Depois da região paralela, a thread principal percorre o vetor e produz o total.

As posições, entretanto, são `long long` contíguos. Em uma máquina com linhas de cache de 64 bytes, oito contadores de 8 bytes ocupam a mesma linha. O vetor foi declarado `volatile` para impedir que os incrementos fossem promovidos para um registrador e, assim, preservar no benchmark os acessos à memória necessários para observar o falso compartilhamento. `volatile` não substitui sincronização; a correção decorre exclusivamente de haver um escritor por posição.

2.3 e 2.4 Substituição por rand_r()

Nas duas últimas versões, cada thread mantém uma semente privada e passa seu endereço a `rand_r()`:

```
int id = omp_get_thread_num();
unsigned int estado = semente + 0x9e3779b9u * (unsigned int)(id + 1);
double x = coordenada(rand_r(&estado));
double y = coordenada(rand_r(&estado));
```

As sementes iniciais são diferentes para que as threads não percorram sequências iguais. Com `schedule(static)`, cada thread processa sempre o mesmo número de pontos e, portanto, as duas versões `rand_r()` produzem exatamente a mesma

contagem. No ambiente POSIX foi usada a implementação da glibc. O fonte inclui uma pequena compatibilidade para a UCRT/MinGW, onde `rand_r()` não existe, mas as medições deste relatório foram feitas no Linux/WSL.

3. Coerência de cache e falso compartilhamento

Cache coerente significa que os núcleos devem concordar sobre o valor de uma linha. Para escrever nela, um núcleo obtém sua propriedade exclusiva e invalida as cópias presentes nos outros caches. Isso é necessário mesmo quando os núcleos alteram palavras diferentes da linha, pois o protocolo opera na granularidade da linha inteira, e não de cada variável.

No contador privado, os incrementos não geram tráfego de coerência. Somente os T subtotais atravessam o `critical` e atualizam `total_global`, ocasionando poucas transferências de propriedade da linha. No vetor contíguo, grupos de oito threads escrevem milhares de vezes em contadores diferentes da mesma linha. A linha “salta” entre os caches, suas cópias são invalidadas repetidamente e os escritores esperam pela propriedade exclusiva. Esse efeito é falso compartilhamento: os dados são logicamente independentes, porém compartilham a unidade física de coerência.

`rand()` introduz outro compartilhamento. Na glibc 2.39 usada no teste, seu estado global é protegido para chamadas concorrentes. As threads disputam essa proteção em cada coordenada — duas vezes por ponto —, serializando o gerador e causando tráfego de coerência em seu estado e em sua trava. Assim, nas duas primeiras versões, o custo de `rand()` domina e esconde parte da diferença entre os métodos de acumulação.

`rand_r()` elimina esse gargalo porque o estado apontado por `estado` é privado. O gerador escala entre os núcleos; por isso, o método de acumulação passa a ser muito mais visível. A versão `rand_r + critical` combina estado aleatório e contador privados durante o laço e só compartilha uma atualização por thread. Já `rand_r + vetor` remove a trava lógica, mas sofre invalidações da linha a cada acerto. Neste caso, evitar `critical` não garante maior desempenho.

Também é importante não generalizar o resultado de `rand()` para toda biblioteca C: o padrão C não oferece `rand_r()` e as escolhas de segurança e estado de `rand()` variam entre implementações. A conclusão quantitativa vale para a glibc medida; a conclusão sobre o vetor contíguo decorre da organização da cache.

4. Resultados

O programa foi compilado no Ubuntu 24.04/WSL 2 com GCC 13.3.0, glibc 2.39 e:

```
gcc -O2 -Wall -Wextra -std=c11 -fopenmp pi_rand_openmp.c -o pi_rand_openmp
```

Foram realizadas cinco execuções com 10.000.000 de pontos, 16 threads e semente 2026 em um AMD Ryzen 7 5700X3D (8 núcleos, 16 threads lógicas). A tabela apresenta a mediana dos tempos:

Versão	π (versão <code>rand_r</code>)	Mediana	Em relação a <code>rand_r + critical</code>
<code>rand + critical</code>	variável entre execuções	0,839048 s	60,81× mais lenta
<code>rand + vetor</code>	variável entre execuções	0,947555 s	68,68× mais lenta
<code>rand_r + critical</code>	3,141424400	0,013797 s	referência
<code>rand_r + vetor</code>	3,141424400	0,038935 s	2,82× mais lenta

Com `rand()`, o vetor foi 12,93% mais lento que o contador privado. Com `rand_r()`, a penalidade do vetor aumentou para 182,20%. Isso não significa que `critical` seja intrinsecamente rápido: ele é barato aqui porque aparece uma vez por thread. Se estivesse dentro do laço, haveria milhões de operações serializadas.

As estimativas de π são plausíveis e próximas do valor real. As versões `rand()` variaram mesmo após `srand(2026)`: a ordem concorrente das chamadas decide quais números consecutivos do estado global formam cada par `(x,y)`. Nas versões `rand_r()`, cada fluxo pertence a uma thread e o escalonamento estático tornou o resultado reprodutível: 7.853.561 acertos nas cinco repetições.

Os valores completos de todas as execuções estão em `resultados.txt`. Tempos muito curtos variam com carga do sistema, frequência do processador e runtime; por isso foi usada a mediana, não uma única observação.

5. Conclusão

A versão mais rápida foi `rand_r + critical`. Seu desempenho resulta de dois tipos de privacidade: cada thread mantém tanto o estado do gerador quanto o contador durante o laço, compartilhando somente o subtotal final. `rand()` foi o principal gargalo nas versões que o utilizaram, devido à serialização do estado global da glibc. No vetor, posições distintas evitaram corrida, mas não evitaram falso compartilhamento; a coerência trabalha com linhas inteiras e tornou a solução sem `critical` mais lenta. Separar cada contador por uma linha de cache (*padding*) ou acumular localmente e gravar no vetor apenas uma vez eliminaria grande parte dessa penalidade, embora mudasse o experimento solicitado.

6. Código-fonte completo

pi_rand_openmp.c · 224 linhas

```
1  /*
2  * Tarefa 8 - Monte Carlo, rand()/rand_r() e falso compartilhamento
3  *
4  * Linux/WSL:
5  * gcc -O2 -Wall -Wextra -std=c11 -fopenmp pi_rand_openmp.c -o pi_rand_openmp
6  * ./pi_rand_openmp [pontos] [threads] [semente]
7  */
8
9  #define _POSIX_C_SOURCE 199506L
10
11  #include <math.h>
12  #include <omp.h>
13  #include <stdio.h>
14  #include <stdlib.h>
15
16  #define MAX_THREADS 256
17  #define CACHE_LINE 64
18
19  /* Total solicitado para as versoes que usam critical. */
20  static long long total_global;
21
22  /* Posicoes distintas, mas intencionalmente contiguas na mesma linha de cache. */
23  _Alignas(CACHE_LINE) static volatile long long acertos_por_thread[MAX_THREADS];
24
25  /* rand_r nao existe na UCRT/MinGW. Esta implementacao de compatibilidade
26  * permite compilar no Windows; no Linux, usa-se a funcao POSIX da libc. */
27  #ifdef _WIN32
28  static int rand_r(unsigned int *semente)
29  {
30      unsigned int proximo = *semente;
31      unsigned int resultado;
32
33      proximo = proximo * 1103515245u + 12345u;
34      resultado = (proximo / 65536u) % 2048u;
35      proximo = proximo * 1103515245u + 12345u;
36      resultado = (resultado << 10) ^ ((proximo / 65536u) % 1024u);
37      proximo = proximo * 1103515245u + 12345u;
38      resultado = (resultado << 10) ^ ((proximo / 65536u) % 1024u);
39      *semente = proximo;
40      return (int)(resultado % ((unsigned int)RAND_MAX + 1u));
41  }
42  #endif
43
44  typedef struct {
45      long long acertos;
46      double segundos;
47  } Resultado;
48
49  static double coordenada(int aleatorio)
50  {
51      return (double)aleatorio / (double)RAND_MAX;
52  }
53
54  /* 1) Contador privado por thread e uma atualizacao global sob critical. */
55  static Resultado rand_critical(long long pontos, unsigned int semente)
56  {
57      Resultado r;
58      double inicio;
59
60      total_global = 0;
61      srand(semente);
62      inicio = omp_get_wtime();
63
64      #pragma omp parallel default(none) shared(pontos, total_global)
65      {
66          long long acertos_locais = 0;
67
68          #pragma omp for schedule(static)
69          for (long long i = 0; i < pontos; i++) {
70              double x = coordenada(rand());
71              double y = coordenada(rand());
72              acertos_locais += (x * x + y * y <= 1.0);
73          }
74
75          #pragma omp critical
76              total_global += acertos_locais;
77      }
78
79      r.segundos = omp_get_wtime() - inicio;
80      r.acertos = total_global;
81      return r;
82  }
```

```

83
84 /* 2) Cada thread incrementa diretamente sua posicao do vetor compartilhado.
85 * volatile preserva os acessos a memoria que evidenciam falso compartilhamento. */
86 static Resultado rand_vetor(long long pontos, unsigned int semente,
87                             int threads)
88 {
89     Resultado r = {0, 0.0};
90     double inicio;
91
92     for (int i = 0; i < threads; i++) {
93         acertos_por_thread[i] = 0;
94     }
95     srand(semente);
96     inicio = omp_get_wtime();
97
98     #pragma omp parallel default(none) shared(pontos, acertos_por_thread)
99     {
100         int id = omp_get_thread_num();
101
102         #pragma omp for schedule(static)
103         for (long long i = 0; i < pontos; i++) {
104             double x = coordenada(rand());
105             double y = coordenada(rand());
106             if (x * x + y * y <= 1.0) {
107                 acertos_por_thread[id]++;
108             }
109         }
110     }
111
112     for (int i = 0; i < threads; i++) {
113         r.acertos += acertos_por_thread[i];
114     }
115     r.segundos = omp_get_wtime() - inicio;
116     return r;
117 }
118
119 /* 3) Igual a (1), mas cada thread possui o estado passado a rand_r. */
120 static Resultado rand_r_critical(long long pontos, unsigned int semente)
121 {
122     Resultado r;
123     double inicio;
124
125     total_global = 0;
126     inicio = omp_get_wtime();
127
128     #pragma omp parallel default(none) shared(pontos, semente, total_global)
129     {
130         int id = omp_get_thread_num();
131         unsigned int estado = semente + 0x9e3779b9u * (unsigned int)(id + 1);
132         long long acertos_locais = 0;
133
134         #pragma omp for schedule(static)
135         for (long long i = 0; i < pontos; i++) {
136             double x = coordenada(rand_r(&estado));
137             double y = coordenada(rand_r(&estado));
138             acertos_locais += (x * x + y * y <= 1.0);
139         }
140
141         #pragma omp critical
142         total_global += acertos_locais;
143     }
144
145     r.segundos = omp_get_wtime() - inicio;
146     r.acertos = total_global;
147     return r;
148 }
149
150 /* 4) Igual a (2), agora com um estado rand_r privado por thread. */
151 static Resultado rand_r_vetor(long long pontos, unsigned int semente,
152                               int threads)
153 {
154     Resultado r = {0, 0.0};
155     double inicio;
156
157     for (int i = 0; i < threads; i++) {
158         acertos_por_thread[i] = 0;
159     }
160     inicio = omp_get_wtime();
161
162     #pragma omp parallel default(none) \
163     shared(pontos, semente, acertos_por_thread)
164     {
165         int id = omp_get_thread_num();
166         unsigned int estado = semente + 0x9e3779b9u * (unsigned int)(id + 1);
167
168         #pragma omp for schedule(static)
169         for (long long i = 0; i < pontos; i++) {
170             double x = coordenada(rand_r(&estado));
171             double y = coordenada(rand_r(&estado));

```

```

172         if (x * x + y * y <= 1.0) {
173             acertos_por_thread[id]++;
174         }
175     }
176 }
177
178 for (int i = 0; i < threads; i++) {
179     r.acertos += acertos_por_thread[i];
180 }
181 r.segundos = omp_get_wtime() - inicio;
182 return r;
183 }
184
185 static void imprimir(const char *nome, Resultado r, long long pontos)
186 {
187     double pi = 4.0 * (double)r.acertos / (double)pontos;
188     printf("%-18s %12lld %.9f %10.6f\n",
189         nome, r.acertos, pi, r.segundos);
190 }
191
192 int main(int argc, char **argv)
193 {
194     long long pontos = argc > 1 ? atoll(argv[1]) : 10000000;
195     int threads = argc > 2 ? atoi(argv[2]) : omp_get_num_procs();
196     unsigned int semente = argc > 3 ? (unsigned int)strtoul(argv[3], NULL, 10)
197         : 2026u;
198     Resultado resultados[4];
199
200     if (argc > 4 || pontos <= 0 || threads <= 0 || threads > MAX_THREADS) {
201         fprintf(stderr, "Uso: %s [pontos] [threads (1..%d)] [semente]\n",
202             argv[0], MAX_THREADS);
203         return EXIT_FAILURE;
204     }
205
206     omp_set_dynamic(0);
207     omp_set_num_threads(threads);
208
209     resultados[0] = rand_critical(pontos, semente);
210     resultados[1] = rand_vetor(pontos, semente, threads);
211     resultados[2] = rand_r_critical(pontos, semente);
212     resultados[3] = rand_r_vetor(pontos, semente, threads);
213
214     printf("Monte Carlo para pi\n");
215     printf("Pontos: %lld | Threads: %d | Semente: %u\n\n",
216         pontos, threads, semente);
217     printf("Versao          Acertos          pi      Tempo (s)\n");
218     imprimir("rand + critical", resultados[0], pontos);
219     imprimir("rand + vetor", resultados[1], pontos);
220     imprimir("rand_r + critical", resultados[2], pontos);
221     imprimir("rand_r + vetor", resultados[3], pontos);
222
223     return EXIT_SUCCESS;
224 }

```